

CS675: Computer Vision Assignment 4

Due date: May 4th at 11:59pm.

Topic: Metric Learning

Assignment Title: Classification with metric learning and Siamese Networks

In this assignment, you will learn to classify images using a metric learning approach based on Siamese networks. Instead of using a traditional classifier like softmax, you will train a model to produce meaningful embeddings, such that images from the same class are embedded closer together than images from different classes.

The classification will be done by comparing a new image embedding to those in the training set and assigning it the class of the closest match. You'll use the [PyTorch Metric Learning toolbox](#) to simplify implementation and focus on learning core concepts. Keras offers a similar toolkit (see [here](#)), which is a valid alternative if you're more comfortable with it. However, the provided code examples use PyTorch.

Before you start, make sure you understand the basics of:

- Metric learning and contrastive loss
- Siamese networks and shared weights
- [Nearest-neighbor classification](#)

Instructions:

Task 1: Implement a PyTorch `Dataset` class for the **CIFAR-10** images that, for each sample, returns a single image tensor and its corresponding class label (an integer from 0–9). Do not generate pairs within the `Dataset` itself: just load and preprocess each image, encode its class index, and hand back (image, label). Create a training and testing split from the dataset to evaluate your model's performance. We will rely on samplers or **miners** in the `DataLoader` to form the actual positive and negative pairs (or triplets) during training. If the dataset is too large for your machine, feel free to use a subset. You can also reuse code from *Assignment 2*, where you already worked with this dataset. Additionally, it's a good idea to include a short snippet that visualizes a few (image, label) pairs to verify that your `Dataset` is correctly mapping each image to its corresponding class.

Task 2: In this task you will design the feature extractor that underlies your Siamese network. Begin by selecting either a small custom CNN (for example, three or four convolutional blocks ending) or a lightweight pre-trained architecture such as [ResNet-18](#). In the second case, do not freeze any layers (you will update every weight during training) and *remember to drop the final classification layer*. In either case, it's crucial to ensure that, given an input image of fixed resolution, the network consistently produces an output tensor of shape `(batch_size, embedding_dim)`. Next, wrap this encoder into a Siamese module by

```
import torch
import torch.nn as nn

class SiameseNetwork(nn.Module):
    def __init__(self, ...):
        super().__init__()
        self.feature_extractor = resnet18(...) # or CustomCNN(...)
        # if using the resnet18 model, make sure to drop the last layer
        # and add a new one with the desired output size

    def forward(self, image):
        return self.feature_extractor(image)

    def compute_siamese_feature(self, image1, image2):
        feature1 = self.forward(image1)
        feature2 = self.forward(image2)
        return feature1, feature2
```

instantiating the network and applying it to a single input, so that your forward pass takes a single image and returns its respective embeddings (other methods can be useful too). Finally, define the contrastive loss function using specified positive and negative margins. These are tunable hyperparameters, but avoid excessive fine-tuning. In most cases, the positive margin can be set to zero: make sure you understand why. Implement this loss by using [ContrastiveLoss](#) from the PyTorch Metric Learning toolbox.

Task 3: To drive effective training, you must integrate two metric-learning sampling strategies:

- First, a simple sampler that draws both positive and negative pairs uniformly. **Tip:** Use `MPerClassSampler`.

```
sampler = samplers.MPerClassSampler(...)

loader = DataLoader(train_dataset, batch_size=..., sampler=sampler)

loss_fn = losses.ContrastiveLoss(pos_margin=..., neg_margin=...)

# 3. Training loop
for x, y in loader:
    x, y = x.to(device), y.to(device)
    emb = model(x) # get the embeddings for the whole batch
    loss = loss_fn(emb, y) # all pairs of already computed embeddings are compared
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

- But do we really need to compare among all pairs? Up to now you have guaranteed that every mini-batch contains positives and negatives (thanks to `MPerClassSampler`). That is *necessary* but not **sufficient** for efficient metric-learning. The miner sub-selects the **most informative pairs/triplets** (e.g. “hard positives” that are still far apart and “hard negatives” that are mistakenly close). You back-prop through just those, which speeds up training, focuses optimization on the mistakes and needs less memory. Without a miner the loss considers all $B \times (B-1)$ positive/negative pairs in the batch. Most of them are *trivial*: positives already close, negatives already far. Their gradients are ~ 0 , so they waste GPU time and dilute the useful signal. In practice, miners are the bridge between how the batch is composed (sampler) and what the loss really cares about (useful distances). Your task is to use `PairMarginMiner` and incorporate it into the existing training loop.

```
sampler = samplers.MPerClassSampler(...)

loader = DataLoader(train_dataset, batch_size=..., sampler=sampler)

miner = miners.PairMarginMiner(...)

loss_fn = losses.ContrastiveLoss(pos_margin=..., neg_margin=...)

# 3. Training loop
for x, y in loader:
    x, y = x.to(device), y.to(device)
    emb = model(x) # get the embeddings for the whole batch
    pairs = miner(emb, y) # get the most informative pairs
    loss = loss_fn(emb, y, pairs) # compute the loss only for the selected pairs
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Task 4: Process the entire training split (or a manageable subset if resources are limited) through each of your trained encoders (the baseline model without a miner and the improved model with a miner) to obtain fixed-length embedding vectors. Using the training embeddings and their labels, fit a Euclidean [kNN](#) classifier ($k = 3, 5$ or 7 is fine) with scikit-learn, then predict the labels for the test embeddings and report standard metrics (overall accuracy, F1 and the confusion matrix). Compare the two strategies side-by-side to quantify how much the miner improves classification and training speed. Finally, apply a dimensionality-reduction technique such as PCA or t-SNE to the train and test embeddings from each model colour-coded by CIFAR-10 class in 2 dimensions, and comment on whether the points form tight, well-separated clusters and if the data distribution is kept between training and test. This is an intuitive visual check that the learned metric spaces are meaningful and not overfitted.

Submission Instructions:

- **Format:** Submit your work as a **Jupyter Notebook (.ipynb)** file, ensuring all code cells are executable. Additionally, export the executed notebook as a **PDF or HTML report**, where all outputs and results are visible.
- **Structure of Submission:** Each task in your assignment should follow this structured approach:
 - a. Introduction and objectives (briefly describe the purpose of the assignment)
 - b. Implementation with justification (explain your solution and justify any design decision)
 - c. Results and analysis (provide good visualizations of the model's predictions)
 - d. Discussion of findings (focus on the interpretation of the learned space)

At the end of the report, provide a conclusion summarizing key findings and reflecting on the effectiveness of the approaches used, highlighting their strengths and limitations.

- e. Conclusions
-

Evaluation Criteria:

Your assignment will be evaluated based on the following criteria:

1. **Introduction and objectives (5%)**
 - Clearly state the purpose of the assignment.
 - Explain what you are doing and why it is important. Provide context or background information if necessary.
2. **Implementation with justification (25%)**
 - Present a well-structured and efficient implementation. Ensure proper documentation and readability of the code.

- Justify every critical decision made in the code. Explain the logic behind your approach, choice of algorithms, tools, or frameworks.
- 3. **Analysis of the results (30%)**
 - Explain the results in relation to the objectives. Present them clearly, using plots, graphs, or explanations as needed.
 - Highlight any patterns, trends, or key insights from the findings.
- 4. **Discussion of findings (40%)**
 - Critically evaluate the results and their implications.
 - Compare findings with expected outcomes (if applicable).
 - Discuss any limitations or challenges faced during the process. Provide suggestions for improvements or future work.

Good luck and have fun!